МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ТАРАСА ШЕВЧЕНКА
Факультет комп'ютерних наук та кібернетики

ЗВІТ

з лабораторної роботи №2
з дисципліни «Розробка мультимедійних систем»

на тему: «Розробка мультимедійного застосунку з використанням сучасних технологій»

Варіант 3 — MIDI-анімація

Виконав: студент групи ПІ-41
Михайло ШЕВНЮК

Київ – 2026

1 МЕТА РОБОТИ

Розробити програму, яка створює анімацію на основі інформації з MIDI-файлу: розібрати структуру MIDI (доріжки, канали, події note_on/note_off, velocity, темп), визначити правила, за якими MIDI-події керують параметрами графічних об’єктів, і наочно продемонструвати зв’язок між MIDI-подіями та поведінкою об’єктів.

2 ПОСТАНОВКА ЗАВДАННЯ ТА ВИКОНАННЯ ВИМОГ

Таблиця 1 – Виконання вимог завдання

Вимога	Як виконано
Анімація створюється на основі інформації з MIDI-файлу	Програма читає будь-який .mid (аргумент командного рядка або перетягування у вікно), переводить події в секунди і за ними будує кожен кадр
MIDI-файл містить щонайменше дві доріжки	Демо-файли: `demo_song.mid` — 6 доріжок (диригентська + Lead, Pad, Arp, Bass, Drums), `mapping_test.mid` — 4 доріжки. Програма перевіряє цю умову і показує попередження, якщо доріжок з нотами менше двох
MIDI впливає щонайменше на два параметри анімації	Реалізовано 9 правил: координата Y, розмір, яскравість, колір/відтінок, форма, поява/ зникнення, довжина «хвоста» й час згасання, тип ефекту для ударних, швидкість руху сцени
Продемонструвати зв’язок між MIDI-подіями та поведінкою об’єктів	Журнал подій у реальному часі («подія MIDI -> дія анімації»), підписи нот біля об’єктів, лінія note_on, піано-рол з курсором, режим фокусу на доріжці, тестовий файл `mapping_test.mid`

3 ОБРАНІ ТЕХНОЛОГІЇ

- **Python 3** — швидка розробка, зрозумілий код, багато бібліотек для мультимедіа.
- **mido** — читання й запис MIDI-файлів (Standard MIDI File). Бібліотека лише розбирає байти файлу на повідомлення; переведення часу в секунди, парування note_on/note_off і всі правила анімації реалізовано власноруч.
- **pygame-ce** (спільнотна версія pygame) — вікно, 2D-графіка з прозорістю, клавіатура, перетягування файлів, відтворення звуку.

- **NumPy** — власний синтезатор звуку (масиви відліків) і швидке створення градієнтів.
- **ffmpeg** (через пакет imageio-ffmpeg) — експорт анімації разом зі звуком у відео MP4 (H.264 + AAC).

Чому не вбудований MIDI-синтезатор: у pygame відтворення MIDI залежить від операційної системи й наявних звукових банків, а позицію відтворення неможливо точно отримати. Власний синтезатор дає однаковий звук на всіх ОС, дозволяє вимикати окремі доріжки і точно синхронізувати звук з анімацією.

4 ТЕОРЕТИЧНІ ВІДОМОСТІ ПРО MIDI

MIDI (Musical Instrument Digital Interface) зберігає не звук, а команди для виконання музики: яку ноту натиснути, з якою силою, коли відпустити, яким інструментом грати.

Формат файлу (SMF). Файл складається із заголовка та доріжок (tracks). Тип 0 — одна доріжка з усіма каналами, тип 1 — кілька паралельних доріжок (зазвичай по одному інструменту на доріжку), тип 2 — незалежні послідовності (рідко).

Канали. Кожне повідомлення адресоване одному з 16 каналів. За стандартом General MIDI канал 10 (у програмі — номер 9, бо нумерація з нуля) зарезервовано для ударних: там номер ноти означає не висоту, а інструмент (36 — бас-барабан, 38 — малий барабан, 42 — закритий хай-хет, 49 — тарілка crash тощо).

Основні події:

- `note_on` (нота, velocity) — натискання клавіші. Velocity 1…127 — сила натискання (гучність).
- `note_off` (нота) — відпускання клавіші. За стандартом `note_on` з velocity = 0 теж означає `note_off`, і багато файлів використовують саме цей запис.

- `program_change` (програма 0…127) — зміна інструмента на каналі. Інструменти General MIDI згруповано в 16 родин по 8 (0–7 фортепіано, 32–39 баси, 88–95 синтезаторні «пади» тощо).
- Мета-події: `track_name` (назва доріжки), `set_tempo` (темп), `time_signature` (розмір такту), `end_of_track`.

Час у MIDI. Кожна подія має дельта-час — кількість тіків (ticks) від попередньої події цієї ж доріжки. Заголовок задає `ticks_per_beat` — скільки тіків в одній чвертній ноті (долі). Темп задається подією `set_tempo` у мікросекундах на чвертну ноту (500 000 мкс = 120 BPM). Отже, на відрізку зі сталим темпом:

```
секунди = тіки / ticks_per_beat × темп_мкс / 1 000 000
BPM = 60 000 000 / темп_мкс
```

Якщо темп змінюється, час рахується по відрізках: секунди до початку відрізка плюс час усередині відрізка за його темпом.

5 АРХІТЕКТУРА ПРОГРАМИ

Таблиця 2 – Модулі програми

Модуль	Призначення
`midi_loader.py`	Читання MIDI: карта темпу (тіки → секунди), парування note_on/note_off, доріжки, події для журналу, перевірка «≥ 2 доріжки»
`mapping.py`	Усі правила відображення MIDI → параметри анімації як окремі невеликі функції
`scene.py`	Обчислення і малювання кадру для моменту t: сцена, легенда, журнал подій, піано-рол, верхня панель
`synth.py`	Власний синтезатор: адитивний синтез з ADSR-обвідною для мелодичних інструментів, шумові ударні, зведення в стерео
`app.py`	Точка входу: інтерактивний режим, керування, синхронізація звуку, експорт у відео, скріншоти
`make_demo_midi.py`	Створення демонстраційних MIDI-файлів
`tests/`	32 автоматичні тести (unittest)
`tools/make_report_assets.py`	Генерація ілюстрацій для звіту

Потік даних:

```
файл .mid --mido--> повідомлення з дельта-тіками
            --midi_loader--> Song: доріжки (Voice) з нотами в секундах, карта темпу, події
```

```
          --scene + mapping--> кадр для моменту t (координати, розміри,
кольори, ефекти)
             --synth--> звук кожної доріжки (stem) --> стерео-мікс -->
pygame.mixer / ffmpeg
```

Ключова ідея — **анімація без стану**: кадр для моменту t повністю обчислюється з MIDI-даних і самого t, а не накопичується кадр за кадром. Тому пауза, перемотування, вимкнення доріжок і експорт у відео працюють однаково і детерміновано (той самий t завжди дає той самий кадр).

6 РОЗБІР MIDI-ФАЙЛУ

Спочатку з усіх доріжок збираються події `set_tempo` (у файлах типу 1 вони зазвичай у «диригентській» доріжці 0) і будується карта темпу:

```
class TempoMap:
    """Перетворення тіків у секунди з урахуванням усіх змін темпу."""

    def _span(self, ticks: int, tempo: int) -> float:
        # тіки -> біти -> мікросекунди -> секунди
        return ticks / self.tpb * tempo / 1_000_000

    def to_seconds(self, tick: int) -> float:
        i = bisect.bisect_right(self.ticks, tick) - 1
             return self.seconds[i] + self._span(tick - self.ticks[i],
self.tempos[i])
```

Для кожного відрізка сталого темпу заздалегідь обчислено, скільки секунд минуло до його початку (`self.seconds`), тож перетворення — це пошук відрізка двійковим пошуком і одна формула.

Потім кожна доріжка проходиться подія за подією, абсолютний час у тіках накопичується з дельта-часів, а натискання парується з відпусканням тієї самої ноти на тому самому каналі:

```
elif msg.type == "note_on" and msg.velocity > 0:
    voice(msg.channel)
         active.setdefault((msg.channel, msg.note), []).append((tick,
msg.velocity))
elif msg.type == "note_off" or (msg.type == "note_on" and msg.velocity
== 0):
    stack = active.get((msg.channel, msg.note))
    if not stack:
        continue                                  # note_off без note_on —
ігноруємо
    on_tick, vel = stack.pop(0)                  # FIFO для однакових нот,
що перекриваються
    voice(msg.channel)["notes"].append(
        Note(sec(on_tick), sec(tick), msg.note, vel, msg.channel))
```

Також обробляються: назви доріжок (`track_name`), інструмент (`program_change`), ноти без `note_off` (закриваються в кінці доріжки), файли типу 0 (ноти різних каналів однієї доріжки розділяються на окремі логічні доріжки).

Правильність перевірено порівнянням з еталонним відліком часу бібліотеки mido: для всіх 628 нот `demo_song.mid` і 112 нот `mapping_test.mid` максимальна розбіжність часу початку — 1 мкс (похибка округлення).

7 ПРАВИЛА ВІДОБРАЖЕННЯ MIDI → АНІМАЦІЯ

Таблиця 3 – Правила відображення MIDI → анімація

MIDI-параметр	Параметр анімації	Правило	Функція в `mapping.py`
Доріжка	Колір	Палітра з 9 кольорів за номером доріжки (збігається з легендою і піано-ролом)	`track_color`
Програма (інструмент GM)	Форма об’єкта	16 родин GM → коло, зірка, квадрат, трикутник, ромб, шестикутник (фортепіано — коло, бас — ромб, пади й струнні — шестикутник, перкусія — зірка)	`program_to_shape`
Висота ноти	Координата Y і відтінок	Від найнижчої до найвищої ноти файлу лінійно знизу вгору сцени; відтінок кольору доріжки зсувається на ±18°	`pitch_to_y`, `note_color`
Velocity	Розмір і яскравість	Радіус r = 5 + 25·(v/127)^1.2 пікселів; яскравість кольору 0.35…1.0 (множник RGB)	`velocity_to_size`, `velocity_to_brightness`
note_on	Поява об’єкта	Об’єкт з’являється на лінії «note_on», «вистрибує» (масштаб 0 → 1.25 → 1.0 за 0.2 с), розходиться кільце	`pop_scale`
Тривалість ноти	Довжина «хвоста» і час згасання	Хвіст тягнеться від об’єкта до лінії note_on, доки нота звучить, тож його довжина = тривалість × швидкість сцени; згасання триває 0.8 + 0.8·d секунд (від 0.8 до 2.5 с)	`release_time`

MIDI-параметр	Параметр анімації	Правило	Функція в `mapping.py`
note_off	Зникнення	Непрозорість спадає до 0, розмір — до 55% за час згасання	`release_time`
Ударні (канал 10)	Тип ефекту	kick — пульсація фону і хвиля знизу; snare/clap — вибух 18 частинок; hi-hat/ride — іскри вгорі; crash — спалах і велике кільце; tom — кільця. Сила ефекту ∝ velocity	`drum_kind`, `DRUM_EFFECTS`
Темп (set_tempo)	Швидкість руху сцени	Об’єкти і сітка долей рухаються ліворуч зі швидкістю 2·BPM пікселів/с (100 BPM → 200 px/s, 124 BPM → 248 px/s)	`tempo_to_scroll_speed`

Додатково: доки нота звучить, об’єкт злегка пульсує (±5% розміру з частотою 3 Гц); під час фокусу на доріжці інші доріжки приглушуються.



Рисунок 1 – Форма залежить від інструмента (родини GM), розмір і яскравість — від velocity

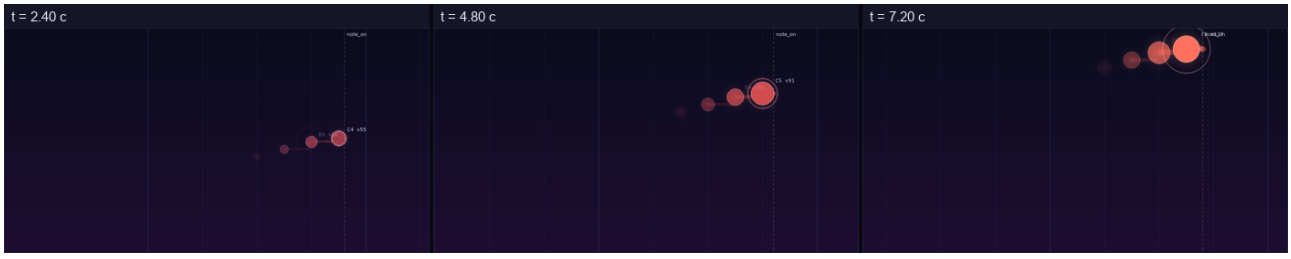


Рисунок 2 – Висхідна гама з crescendo у mapping_test.mid: кожна наступна нота вища — об’єкт вище; гучніша — більший

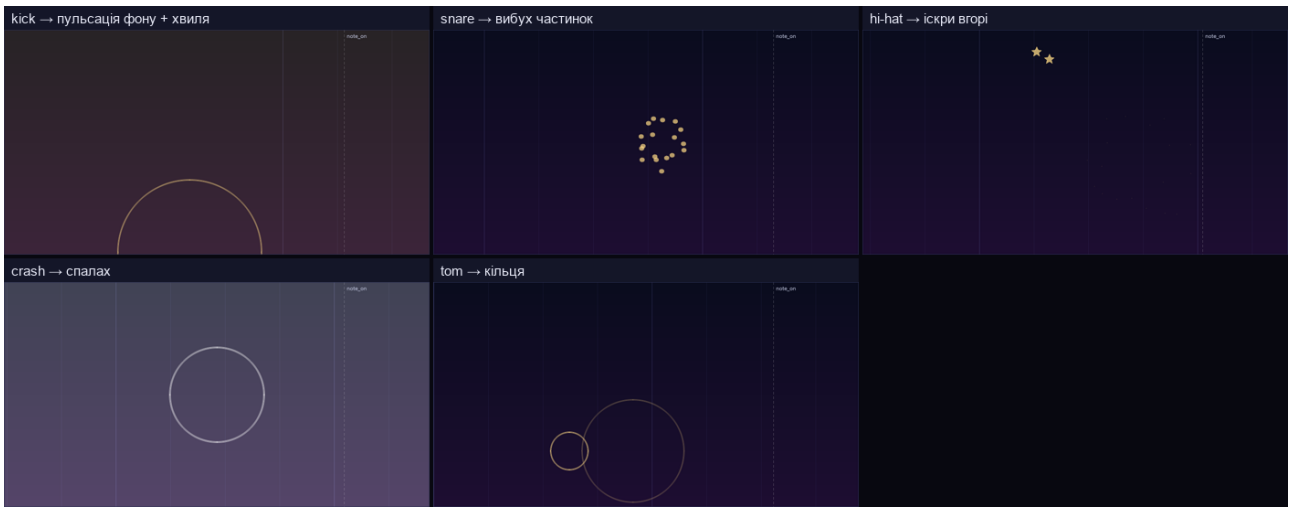


Рисунок 3 – Ефекти ударних інструментів (канал 10)

Положення об’єкта по горизонталі обчислюється через «пройдений шлях» сцени — інтеграл швидкості за часом. Оскільки швидкість залежить від темпу, а темп змінюється стрибками, інтеграл — це сума по відрізках сталого темпу:

```
def scroll(self, t: float) -> float:
    i = max(bisect.bisect_right(self.seg_t, t) - 1, 0)
    return self.seg_s[i] + (t - self.seg_t[i]) * self.seg_v[i]
```

Тоді координата об’єкта: `x = spawn_x - (scroll(t) - scroll(note.start))` — нота стартує на лінії note_on і відходить ліворуч рівно на стільки, скільки «проїхала» сцена з моменту її початку. Фрагмент обчислення всіх видимих об’єктів:

```
for n in v.notes[lo_i:hi_i]:
    rel = mp.release_time(n.duration)
    if t > n.end + rel:
        continue
        # Об'єкт з'являється на лінії spawn_x і рухається ліворуч зі швидкістю сцени.
    x = L.spawn_x - (now - self.scroll(n.start))
    tail_x = L.spawn_x - (now - self.scroll(min(t, n.end)))
    age = t - n.start
    held = t < n.end
    fade = 1.0 if held else max(0.0, 1 - (t - n.end) / rel) ** 1.5
      r = mp.velocity_to_size(n.velocity) * mp.pop_scale(age) * (0.55 + 0.45 * fade)
```

Щоб не перебирати всі ноти на кожному кадрі, початки нот кожної доріжки зберігаються відсортованими, і двійковим пошуком (`bisect`) відбираються лише ті, що можуть бути видимими в момент t.

8 ЗВУК І СИНХРОНІЗАЦІЯ

Звук синтезується власним кодом (`synth.py`):

- **мелодичні інструменти** — адитивний синтез: сума синусоїд-гармонік, частота ноти f = 440 · 2^((n − 69)/12). Набір гармонік і обвідна залежать від родини інструмента: «щипкові» (фортепіано, гітара), «дзвіночки» (music box), орган, бас, пад (два трохи розстроєні генератори — ефект хорусу), лід (прямокутна хвиля з вібрато). Гармоніки вище частоти Найквіста не генеруються;
- **ADSR-обвідна** — Attack (наростання), Decay (спад), Sustain (рівень утримання), Release (згасання після note_off);
- **ударні** — бас-барабан: синусоїда з частотою, що падає від 140 до 45 Гц; малий барабан: шум + тон 185 Гц; хай-хет і тарілки: високочастотний шум з різним затуханням; томи — синусоїди різної висоти;
- кожна доріжка рендериться окремо (stem), тому при вимкненні доріжки клавішами 1–9 вона одночасно зникає з екрана і з міксу; зведення в стерео — з рівнопотужним панорамуванням, пікова гучність нормалізується до −1 dBFS.

Синхронізація: звук і годинник анімації запускаються одночасно; час кадру = позиція, з якої почалося відтворення, + час, що минув від старту. Пауза зупиняє і звук, і годинник; перемотування перезапускає звук з нової позиції:

```
def position(self) -> float:
        t = self.offset + (time.perf_counter() - self.started if self.started is not None else 0.0)
    return min(t, self.total)
```

При експорті у відео кадри рендеруються з точним кроком 1/fps і передаються в ffmpeg разом із тим самим синтезованим звуком, тому у відео звук і зображення збігаються покадрово.

9 ІНТЕРФЕЙС І КЕРУВАННЯ

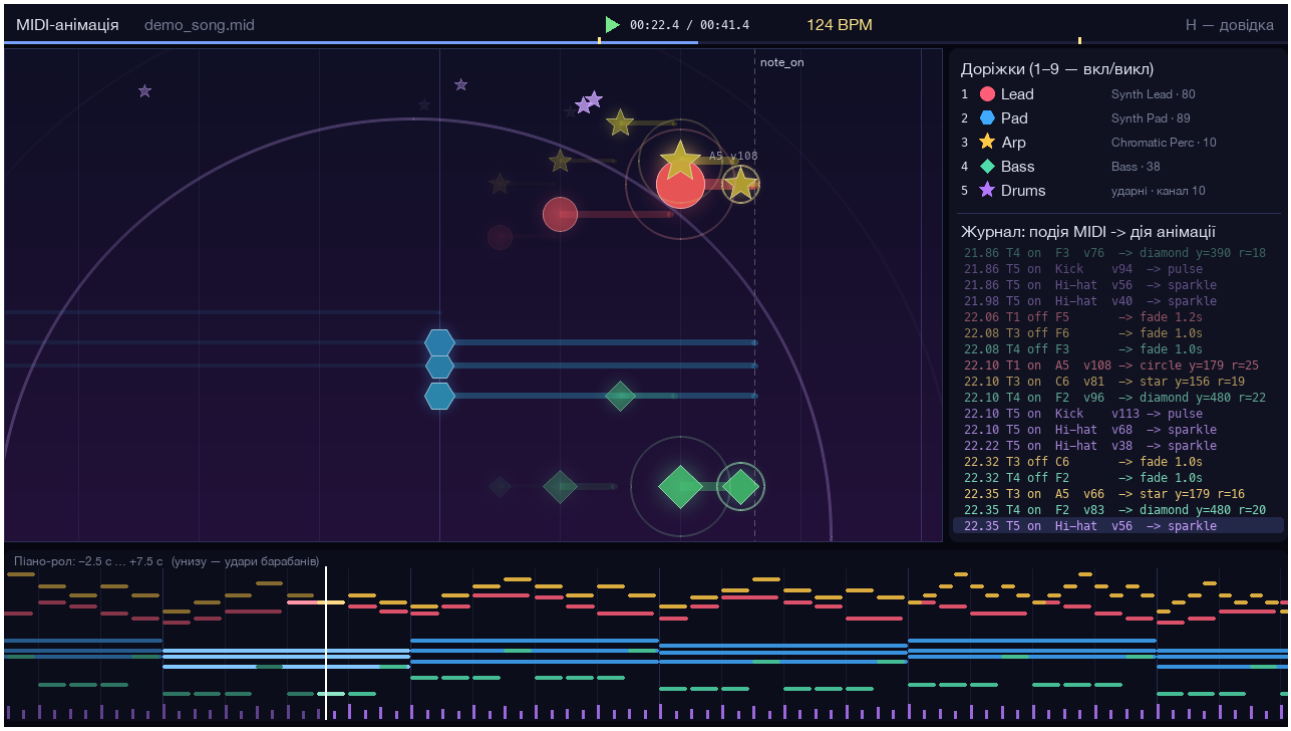


Рисунок 4 – Головне вікно: сцена з анімацією, легенда доріжок, журнал подій, піано-рол

Елементи вікна:

- **верхня панель** — назва файлу, стан відтворення, поточний і загальний час, поточний темп (BPM), смуга прогресу з жовтими позначками змін темпу;
- **сцена** — анімовані об’єкти, сітка долей (яскравіші лінії — початки тактів), пунктирна лінія note_on, де з’являються нові ноти; біля нових нот — підписи «нота + velocity»;
- **легенда доріжок** — номер клавіші, форма і колір, назва, родина інструмента й номер програми;
- **журнал подій** — останні MIDI-події і що вони спричинили в анімації;
- **піано-рол** — ноти всіх доріжок у вікні від −2.5 до +7.5 с навколо курсору (білий вертикальний), окремий ряд для ударів барабанів.

Таблиця 4 – Керування з клавіатури

Клавіша	Дія
Пробіл	Відтворення / пауза

Клавіша	Дія
R	З початку
← / →	Перемотування на 5 с назад / вперед
1–9	Сховати/показати доріжку (і вимкнути/увімкнути її звук)
Tab	Фокус на доріжці: підписи всіх її нот, фільтр журналу, інші доріжки приглушено
L	Підписи нот на сцені
H	Довідка
S	Скріншот у папку `screenshots/`
N / P	Відкрити наступний / попередній файл у тій самій папці
Перетягування .mid у вікно	Відкрити інший MIDI-файл
Esc	Вихід

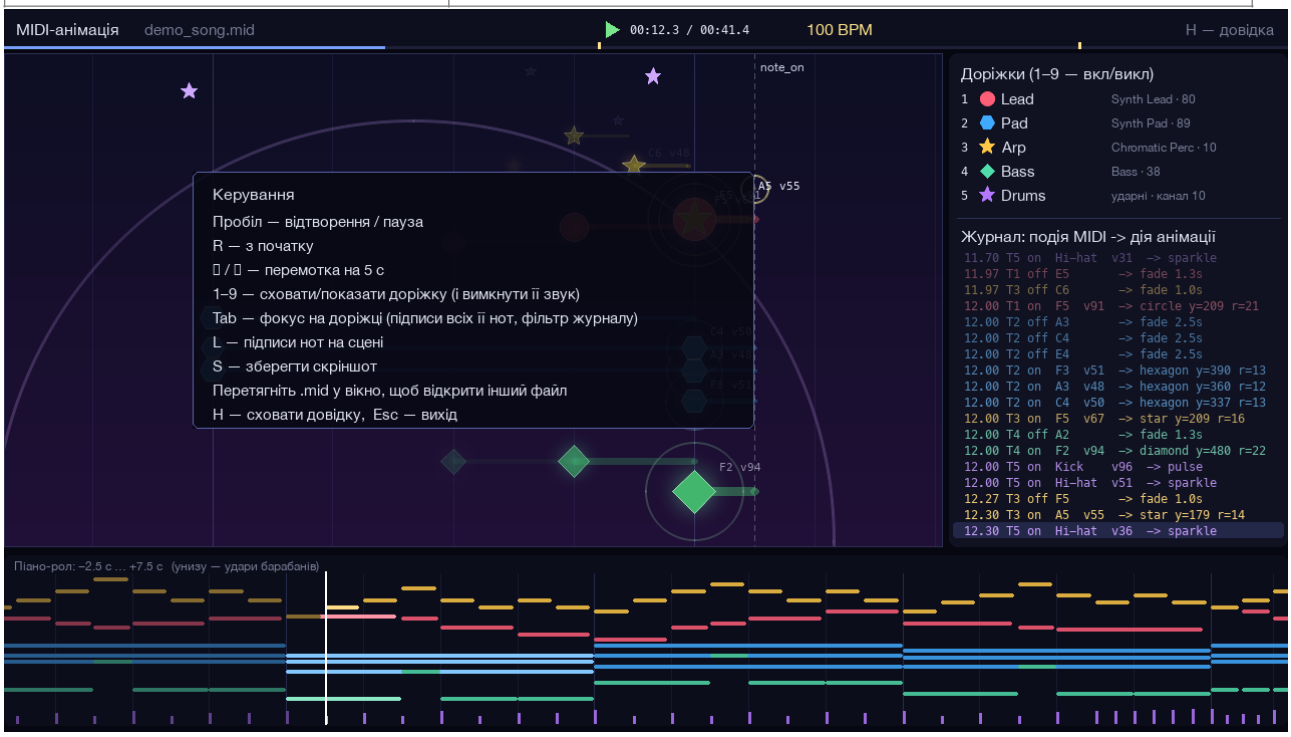


Рисунок 5 – Довідка з керування (клавіша H)

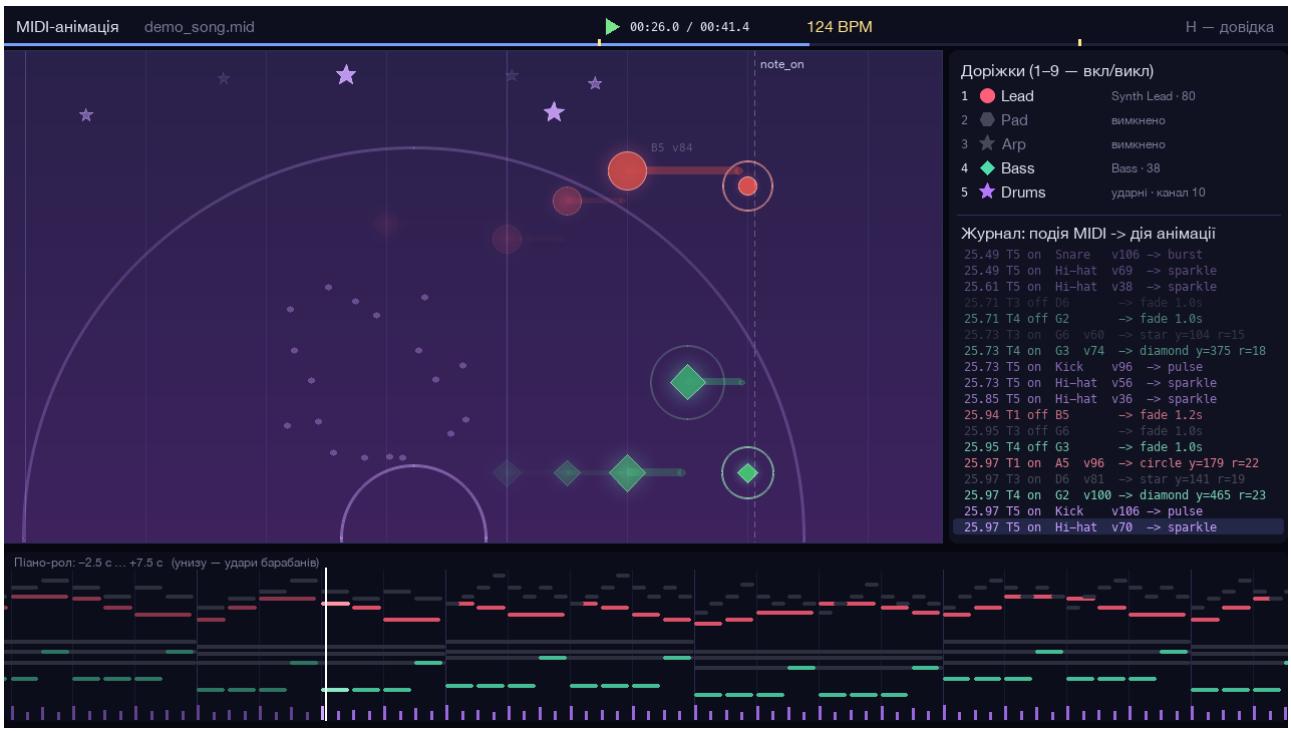


Рисунок 6 – Доріжки 2 (Pad) і 3 (Arp) вимкнено — їх немає на сцені, у звуці, а в піано-ролі вони сірі

10 ДЕМОНСТРАЦІЯ ЗВ’ЯЗКУ MIDI-ПОДІЙ І ПОВЕДІНКИ ОБ’ЄКТІВ

Журнал подій. Кожен рядок — це реальна MIDI-подія з часом і наслідок для анімації. Наприклад, `22.10 T1 on A5 v108 -> circle y=179 r=25` означає: на 22.10 с доріжка 1 (Lead) натиснула ноту A5 з velocity 108, тому з’явилося коло на висоті y = 179 пікселів радіусом 25. `T1 off F5 -> fade 1.2s` — нота відпущена, об’єкт згасає 1.2 с. `tempo 124 BPM -> scroll 248px/s` — змінився темп, сцена прискорилася. Найновіший рядок підсвічено; значення y і r у журналі обчислюються тими самими функціями, що й малювання: y — це висота об’єкта, r — його базовий радіус (на екрані він ще змінюється анімацією появи, легкою пульсацією і зменшенням під час згасання). Зміна інструмента (program change) посеред треку позначається в журналі як «без змін»: форма й тембр доріжки визначаються інструментом її першої ноти. Якщо рядок не вміщується в панель, він друкується дрібнішим шрифтом, а у вузькому вікні обрізається по словах з «…», тож число ніколи не показується обрізаним.



Рисунок 7 – Журнал подій: «подія MIDI -> дія анімації»

Життєвий цикл однієї ноти (нота E4 тривалістю 4 долі з `mapping_test.mid`):



Рисунок 8 – Послідовність кадрів: note_on — поява, утримання — росте хвіст, note_off — згасання

Фокус на доріжці (Tab): видно лише події обраної доріжки, кожна її нота підписана, інші доріжки приглушені — зручно показувати, як конкретна мелодія перетворюється на рух об’єктів.



Рисунок 9 – Фокус на доріжці 1 (Lead): журнал відфільтровано, підписи всіх нот

Тестовий файл `mapping_test.mid` побудовано так, щоб кожне правило було добре видно: висхідна гама C3–C6 з поступовим наростанням velocity від 20 до 127 (об’єкти піднімаються і ростуть), однакова нота E4 різної тривалості (від 1/4 до 4 долей — різна довжина хвоста і час згасання), по такту окремо на кожен ударний інструмент (kick, snare, hi-hat, crash, томи — різні ефекти), зміна темпу 90 → 150 BPM (сцена помітно прискорюється), спадна гама з diminuendo.

Демонстраційне відео `demo/demo.mp4` (41 с, 1280×720, 30 кадрів/с, H.264 + AAC) записано програмою в режимі експорту з файлу `demo_song.mid`: 5 доріжок, 628 нот, темп 100 → 124 → 92 BPM.

11 ТЕСТУВАННЯ

Автоматичні тести (`python -m unittest discover -s tests`), усього 32, усі проходять:

- переведення тіків у секунди за замовчуванням (120 BPM) і після зміни темпу; зміна темпу в окремій «диригентській» доріжці правильно зсуває ноти інших доріжок;

- `note_on` з velocity 0 розпізнається як `note_off`; нота без `note_off` закривається в кінці доріжки;
- розпізнавання ударних (канал 10), інструмента і назви доріжки; попередження для файлу з однією доріжкою; демо-файли мають ≥ 2 доріжки і зміну темпу;
- правила відображення: вища нота — менший Y (вище на екрані), більша velocity — більший розмір і яскравість, довша нота — довше згасання, швидший темп — швидша сцена, правильні форми і ефекти;
- синтезатор: довжина кожного stem дорівнює тривалості пісні + 1.5 с, мікс нормалізований до −1 dBFS, вимкнення доріжок дає тишу, частота A4 = 440 Гц;
- кадри рендеряться без помилок для різних моментів і станів інтерфейсу; у момент удару kick сцена світліша, ніж перед ним;
- некоректні вхідні дані (див. таблицю нижче): кожен зламаний файл дає зрозуміле повідомлення, а файл без нот малюється без помилок;
- граничні випадки: темп 0 у `set_tempo`, файл тривалістю понад 30 хв (пошкоджені часові позначки), папка замість файлу, зміна інструмента посеред треку, рядки журналу у вузькому вікні;
- некоректні параметри командного рядка (`--screenshot abc`, `--hide x`, `--fps 0`, `--end` менший за `--start`) дають повідомлення argparse, а не трасування.

Перевірка на різних і некоректних вхідних даних. Скрипт `tools/make_test_inputs.py` створює в `midi/samples/` 12 файлів. У вікні програми їх можна перебрати клавішею N.

Таблиця 5 – Перевірка на різних і некоректних вхідних даних

Файл	Що в ньому	Реакція програми
01_ok_type0_multichannel	MIDI типу 0: піаніно, бас і ударні в одному треку, на різних каналах	3 окремі доріжки, без попереджень

Файл	Що в ньому	Реакція програми
02_ok_tempo_changes	темп 70 → 140 → 90 BPM	сцена прискорюється і сповільнюється, у журналі події tempo
03_ok_extreme_values	ноти 21 і 108, velocity 1 і 127, ноти в 1 тік, акорд з 12 нот	усі ноти показано, розміри в межах сцени
04_warn_one_track	лише одна доріжка з нотами	відкривається, «Увага: MIDI має менше двох доріжок…»
05_warn_no_notes	коректний файл без жодної ноти	порожня сцена, «Увага: у файлі немає нот»
06_warn_unpaired_notes	2 note_on без note_off, 1 note_off без note_on	ноти закрито в кінці треку, зайвий note_off пропущено, обидва випадки в попередженні
07_bad_empty	0 байт	«файл порожній (0 байт)»
08_bad_not_midi	текст із розширенням .mid	«це не MIDI-файл: на початку немає заголовка MThd»
09_bad_truncated	demo_song.mid, обрізаний посередині	«файл обрізаний: дані закінчились раніше, ніж очікувалось»
10_bad_zero_ticks	ticks_per_beat = 0 у заголовку	«некоректний заголовок: ticks_per_beat = 0»
11_bad_data_byte	байт даних 200 > 127 у note_on	«пошкоджені дані всередині треку»
12_bad_wrong_extension.txt	текстовий файл з розширенням .txt	«це не MIDI-файл: на початку немає заголовка MThd» (тип файлу визначається за вмістом, а не за розширенням)

Помилки бібліотеки mido англійською і часто без тексту (наприклад, `EOFError`). Функція `open_midi` у `midi_loader.py` перехоплює їх і перетворює на `MidiLoadError` з поясненням українською. Якщо файл не відкрився, програма показує червоне повідомлення з причиною і продовжує відтворювати попередній файл. Якщо зламаний файл передано в командному рядку, відкривається демо. У режимі без вікна (`--export`, `--screenshot`) виводиться повідомлення про помилку з кодом виходу 1, а некоректні параметри (`--size`, `--fps`, `--hide`, `--screenshot`, `--start`/`--end`) відхиляються з поясненням. Крім 12 файлів із таблиці, програма відхиляє з поясненням: `set_tempo` з темпом 0, файли довші за 30 хв (так виглядають пошкоджені часові позначки, через які інакше синтез звуку зайняв би десятки гігабайтів пам’яті), час у форматі SMPTE і

папку замість файлу. Експорт файлу без нот не запускається: «у файлі немає нот — експортувати нічого». Будь-яка непередбачена помилка під час відкриття файлу у вікні також показується червоним повідомленням, і програма працює далі.

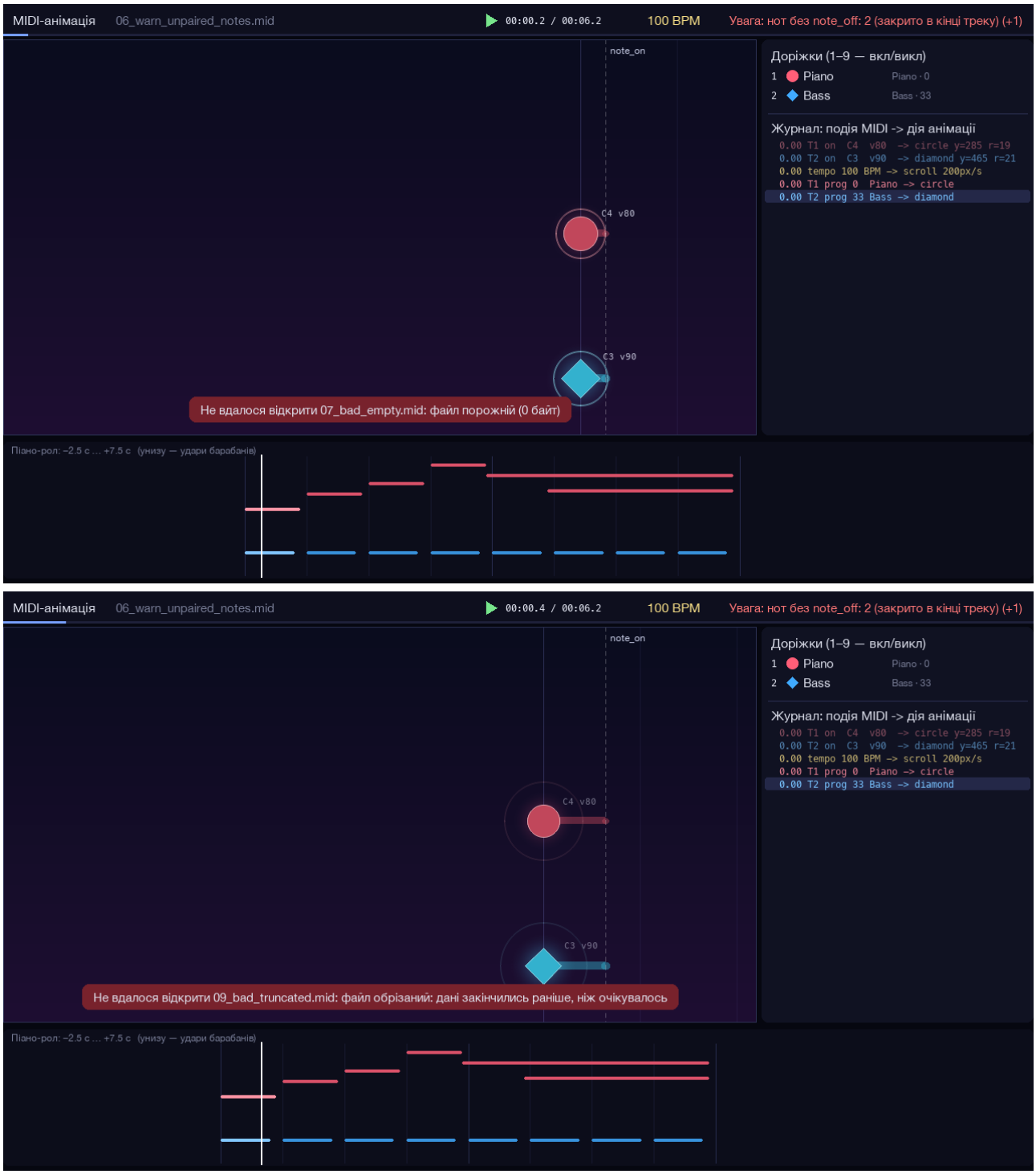


Рисунок 10 – Файл із попередженнями (ноти без пари) грає далі, а спроби відкрити зламані файли (угорі — порожній, унизу — обрізаний) дають червоні повідомлення з причиною

Також перевірено інтерактивний режим зі сценарієм натискань (пауза, перемотування, вимкнення доріжки, фокус, довідка, скріншот, відкриття іншого файлу перетягуванням, відмова відкрити не-MIDI файл). Швидкодія: кадр 1280×720 малюється за 1.3–2.2 мс, тобто 60 кадрів/с із великим запасом; експорт 41-секундного відео займає близько 9 с.

12 ДОДАТКОВІ МОЖЛИВОСТІ

- власний синтезатор звуку з різними тембрами за родинами інструментів і синхронне відтворення;
- вимкнення окремих доріжок одночасно на екрані і в звуці;
- журнал подій «MIDI → анімація», підписи нот, режим фокусу на доріжці;
- піано-рол з «майбутніми» нотами і курсором;
- перемотування, пауза, перезапуск, довідка, скріншоти;
- відкриття будь-якого MIDI-файлу перетягуванням у вікно або клавішами N / P, підтримка типів 0 і 1, кількох змін темпу;
- перевірка вхідних даних: зрозумілі повідомлення про пошкоджені файли і попередження про проблеми в даних, набір із 12 тестових файлів;
- експорт анімації разом зі звуком у відео MP4;
- генератор демонстраційних MIDI-файлів і автоматичні тести.

13 ВИСНОВКИ

У роботі розроблено програму, яка перетворює MIDI-файл на синхронізовану зі звуком анімацію. Розібрано структуру MIDI: доріжки й канали, події note_on/note_off (включно з note_on з нульовою velocity), program_change, а також систему часу на основі тіків, ticks_per_beat і темпу. Реалізовано точне перетворення часу з урахуванням змін темпу — результат збігається з еталонним відліком mido з точністю до 1 мкс.

Сформульовано і реалізовано 9 правил відображення: висота ноти керує вертикальним положенням і відтінком, velocity — розміром і яскравістю, інструмент — формою, доріжка — кольором, тривалість — довжиною хвоста і

часом згасання, note_on/note_off — появою і зникненням, ударні — типом ефекту, темп — швидкістю руху сцени. Зв’язок між подіями і поведінкою об’єктів продемонстровано журналом подій, підписами нот, піано-ролом і спеціальним тестовим MIDI-файлом.

Підхід «кадр як функція часу» спростив реалізацію паузи, перемотування та експорту у відео і зробив результат детермінованим, що дозволило перевірити програму автоматичними тестами.